

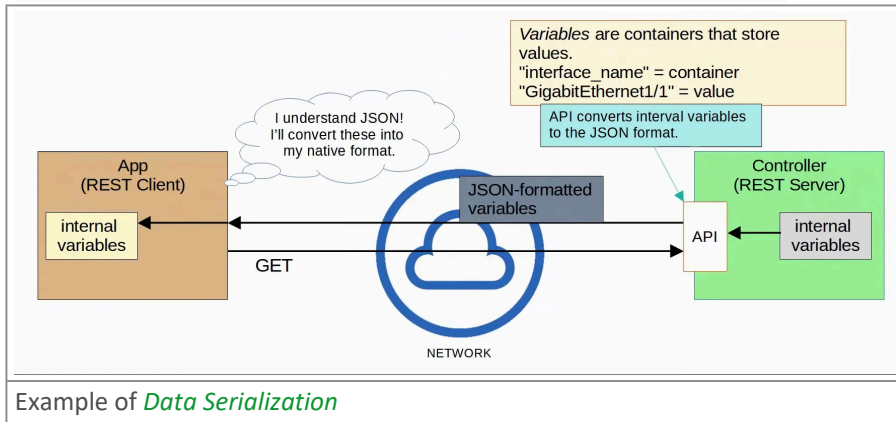
Day 60 - JSON, XML, & YAML

Wednesday, September 9, 2026 8:37 PM

JSON, XML, & YAML Info

Data serialization is the process of converting data into a standardized format/structure that can be stored (in a file) or transmitted (over a network) and reconstructed later (by a different application).

- This allows the data to be communicated between applications in a way both applications can understand.
- **Data serialization** languages allow us to represent **variables** with text.



Data Serialization Languages

There are three different **data serialization** languages you need to know for the CCNA, being:

1. **JSON**
2. **XML**
3. **YAML**

1. JSON

JavaScript Object Notation (JSON) is an open standard file format and data interchange format that uses human-readable text to store and transmit data objects.

- It was derived from **JavaScript**, but it is language-independent and many modern programming languages are able to generate and read **JSON** data.

JSON Properties

- **JSON** is very human-readable
- **REST APIs** often use **JSON**.
- **Whitespace** is insignificant
- **JSON** can represent four 'primitive' data types

1. String

- A **string** is a text value. It is surrounded by double quotes " ".
- Examples: ("Hello." | "five" | "5" | "true" | "null")

2. Number

- A **number** is a numeric value. It is not surrounded by quotes.
- Examples: (5 | 100)

3. Boolean

- A **boolean** is a data type that has only two possible values. It is not surrounded by quotes.
- (true | false)

4. Null

- A **null** value represents the intentional absence of any object value. It is not surrounded by quotes.
- (null)

- **JSON** also has two 'structured' data types:

1. Object

- Sometimes called a **dictionary**
- An **object** is an unordered list of **key-value** pairs (**variables**)
- The **key** is a **string**.
- The **value** is any valid **JSON** data type (**string, number, boolean, null, object, array**).
- The **key** and **value** are enclosed with curly brackets (**{ }**)
- The **key** and **value** are separated by a colon (**:**)
- If there are multiple **key-value** pairs, each pair is separated by a comma (**,**)
- **Objects** within **objects** are called 'nested **objects**'.

2. Array

- Values are enclosed with square brackets (**[]**)
- An **array** is a series of **values** separated by commas (**,**)
- The **values** can be any type of data type.
- The **values** don't have to be the same data type.

```
{
  "interface": "GigabitEthernet1/1",
  "is_up": true,
  "ipaddress": "192.168.1.1",
  "netmask": "255.255.255.0",
  "speed": 1000
}
```

```
{"interface": "GigabitEthernet1/1", "is_
up": true, "ipaddress": "192.168.1.1", "ne
tmask": "255.255.255.0", "speed": 1000}
```

JSON Object Example (1)

```
{
  ↓string      ↓string
  "interface": "GigabitEthernet1/1",
  ↓string      ↓boolean
  "is_up": true,
  ↓string      ↓string
  "ipaddress": "192.168.1.1",
  ↓string      ↓string
  "netmask": "255.255.255.0",
  ↓string      ↓number
  "speed": 1000
}
```

JSON Object Example (2)

```
{
  key      value (object)
  "device": {
    "name": "R1",
    "vendor": "Cisco",
    "model": "1101"
  },
  key      value (object)
  "interface config": {
    "interface_name": "GigabitEthernet1/1",
    "is_up": true,
    "ipaddress": "192.168.1.1",
    "netmask": "255.255.255.0",
    "speed": 1000
  }
}
```

JSON Object Example (3)

```
{
  "interfaces": [
    "GigabitEthernet1/1",
    "GigabitEthernet1/2",
    "GigabitEthernet1/3"
  ],
  "random_values": [
    "Hi",
    5
  ]
}
```

JSON Array Example (4)

2. XML

Extensible Markup Language (XML) was developed as markup language, but is now used as a general *data serialization* language.

XML Properties

- **XML** is generally less human-readable than **JSON**.
- **REST APIs** often use **XML**.
- *Whitespace* is insignificant.
- **XML** is represented as *key-value* pairs.
- It is presented as `<key>value</key>`.

```
R1#show ip interface brief | format
<?xml version="1.0" encoding="UTF-8"?>
<ShowIpInterfaceBrief xmlns="ODM://built-in/show_ip_interface_brief">
  <SpecVersion>built-in</SpecVersion>
  <IPInterfaces>
    <entry>
      <Interface>GigabitEthernet0/0</Interface>
      <IP-Address>192.168.1.1</IP-Address>
      <OK>YES</OK>
      <Method>manual</Method>
      <Status>up</Status>
      <Protocol>up</Protocol>
    </entry>
    <entry>
      <Interface>GigabitEthernet0/1</Interface>
      <OK>YES</OK>
      <Method>unset</Method>
      <Status>administratively down</Status>
      <Protocol>down</Protocol>
    </entry>
  </IPInterfaces>
</ShowIpInterfaceBrief>
```

XML Example (1)

3. YAML

Yet Another Markup Language (YAML) is also used as a *data serialization* language.

- **YAML** is used by the network automation tool **Ansible**.

YAML Properties

- **YAML** is the most human-readable language.
- *Whitespace* is significant (unlike **JSON** and **XML**). That means that indentation is very important.
- **YAML** files start with `---`.
- `-` is used to indicate a list.

- *Keys* and *values* are represented as *key:value*

```
---
ip_interfaces:
- Interface: GigabitEthernet0/0
  IP-Address: 192.168.1.1
  OK?: 'YES'
  Method: manual
  Status: up
  Protocol: up
- Interface: GigabitEthernet0/1
  IP-Address: unassigned
  OK?: 'YES'
  Method: unset
  Status: administratively down
  Protocol: down
```

YAML Example (1)